Lab 9: File Systems

Course:	COSC315 - Operating Systems
Duration:	1.5 Hours
Prerequisites:	Labs 1-8 completed
Total Marks:	10

1. Lab Overview

This lab explores file systems - how operating systems organize, store, and retrieve data on storage devices. Students will work with file operations using system calls, explore directory structures, simulate disk allocation methods, and understand how files are represented at the OS level. These concepts are essential for understanding how data persistence works in operating systems.

2. Learning Objectives

By the end of this lab, students will be able to:

- Use low-level file system calls (open, read, write, close, lseek)
- Understand file descriptors and their relationship to open files
- Navigate and manipulate directory structures programmatically
- Retrieve and interpret file metadata using stat()
- Simulate disk allocation methods (contiguous, linked, indexed)
- Compare trade-offs between different allocation strategies

3. Background Concepts

3.1 File System Calls

Unix/Linux provides low-level system calls for file operations:

System Call	Description
`open()`	Opens a file, returns file descriptor
`read()`	Reads bytes from file descriptor into buffer
`write()`	Writes bytes from buffer to file descriptor
`close()`	Closes file descriptor
`lseek()`	Repositions file offset (for random access)
`stat()`	Retrieves file metadata (size, permissions, timestamps)

3.2 File Descriptors

A **file descriptor** is a small non-negative integer that identifies an open file. Standard file descriptors:

- **0:** Standard Input (stdin)
- **1:** Standard Output (stdout)
- **2:** Standard Error (stderr)

3.3 Disk Allocation Methods

Method	Advantages	Disadvantages
Contiguous	Fast sequential & random access	External fragmentation, file growth difficult
Linked	No external fragmentation, easy growth	Slow random access, pointer overhead

Indexed	Fast random access, no ext. fragmentation	Index block overhead, max file size limit

4. Lab Tasks

Create a directory for this lab: `mkdir -p ~/cosc315/lab9 && cd ~/cosc315/lab9`

Task 1: Low-Level File Operations (2 marks)

Objective: Use system calls to perform file operations without standard library functions.

Instructions:

1. Create a file named file_ops.c
2. Implement the following using ONLY system calls (no fopen, fread, etc.):
   a) Create and write to a new file
   b) Close and reopen the file for reading
   c) Read and display the contents
   d) Use lseek() to read from a specific position
3. Include proper error handling for all system calls
4. Print the file descriptor number for each open file

Required System Calls:

```
#include <fcntl.h>    // open() flags
#include <unistd.h>   // read(), write(), close(), lseek()

int fd = open("file.txt", O_CREAT | O_WRONLY, 0644);
write(fd, buffer, length);
lseek(fd, offset, SEEK_SET);  // SEEK_SET, SEEK_CUR, SEEK_END
```

✓ **Checkpoint 1:** File created, written, read back successfully using only system calls.

Task 2: File Metadata with stat() (2 marks)

Objective: Retrieve and display comprehensive file metadata.

Instructions:

5. Create a file named file_stat.c
6. Accept a filename as command-line argument
7. Use stat() to retrieve and display:
   e) File type (regular, directory, symlink, etc.)
   f) File size in bytes
   g) Number of hard links
   h) Inode number
   i) Permissions (in octal and symbolic form)
   j) Last access, modification, and status change times
8. Test with different file types (regular file, directory, /dev/null)

Useful Macros:

```
S_ISREG(mode)  // Is regular file?
S_ISDIR(mode)  // Is directory?
S_ISLNK(mode)  // Is symbolic link? (use lstat)
```

✓ **Checkpoint 2:** Program displays all metadata fields correctly for different file types.

Task 3: Directory Traversal (2 marks)

Objective: Programmatically read and display directory contents.

Instructions:

9. Create a file named dir_list.c
10. Accept a directory path as command-line argument (default to ".")
11. Use opendir(), readdir(), closedir() to list contents
12. For each entry, display:
    k) Entry name
    l) Entry type (file, directory, symlink, etc.)
    m) Size (for regular files)
13. Sort entries alphabetically
14. Display total count of files and directories

Code Structure:

```
#include <dirent.h>

DIR *dir = opendir(path);
struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    printf("%s\n", entry->d_name);
}
closedir(dir);
```

✓ **Checkpoint 3:** Directory listing shows type and size for each entry.

Task 4: Disk Allocation Simulator (2 marks)

Objective: Simulate and compare contiguous, linked, and indexed allocation methods.

Instructions:

15. Create a file named disk_alloc.c
16. Simulate a disk with 100 blocks
17. Implement three allocation methods:
    n) **Contiguous:** Store start block and length
    o) **Linked:** Each block points to next block (-1 for end)
    p) **Indexed:** Index block contains list of data blocks
18. Create files of different sizes (5, 10, 20 blocks)
19. Simulate reading block N of each file and count disk accesses
20. Display the disk allocation map for each method

Expected Comparison:

Method	Read Block N	Disk Accesses
Contiguous	Direct calculation	1
Linked	Follow chain	N (must traverse)
Indexed	Read index + data	2

✓ **Checkpoint 4:** Simulator correctly counts disk accesses for each allocation method.

Task 5: Simple File System Simulation (1 mark)

Objective: Create a simple in-memory file system with basic operations.

Instructions:

21. Create a file named simple_fs.c
22. Implement a simple file system with:
    q) A fixed number of inodes (e.g., 16)
    r) A fixed number of data blocks (e.g., 64)
    s) A free block bitmap
23. Support these operations:
    t) `create(name)`: Create a new file
    u) `delete(name)`: Delete a file
    v) `write(name, size)`: Allocate blocks for file
    w) `list()`: List all files
24. Display bitmap and inode table after operations

✓ **Checkpoint 5:** File system correctly manages inodes and blocks.

Task 6: Conceptual Questions (1 mark)

Answer the following questions in your submission document:

Q1: Explain why file descriptors 0, 1, and 2 are reserved. What would happen if a program closes fd 1 and then opens a new file?

Q2: Compare the inode-based file system (Unix) with the FAT file system (Windows). What are the key structural differences?

Q3: Why does linked allocation perform poorly for random access? In what scenario would linked allocation be a good choice?

Q4: What is the purpose of a free-space bitmap? How does it help with allocation efficiency?

5. Submission Requirements

Submit a single PDF document with the following structure:

Section	Content Required	Notes
Cover Page	Name, Student ID, Lab Number, Date	1 page
Task 1	Code + output showing file operations	Show file descriptors
Task 2	Code + output for different file types	Test file, dir, /dev/null
Task 3	Code + directory listing output	Show type and size
Task 4	Code + allocation comparison	Compare disk accesses
Task 5	Code + file system operations	Show bitmap changes
Task 6	Written answers to all 4 questions	2-4 sentences per question

File Naming Convention: Lab9_StudentID_LastName.pdf

6. Marking Schema

Criteria	Marks	Requirements for Full Marks
Task 1: File Operations	2	System calls only, proper error handling
Task 2: File Metadata	2	All metadata fields, multiple file types
Task 3: Directory Traversal	2	Lists entries with type and size
Task 4: Disk Allocation	2	All three methods, correct access counts
Task 5: Simple File System	1	Basic operations work correctly
Task 6: Conceptual Questions	1	Accurate, thoughtful answers
TOTAL	**10**	